

Number: P3748R0

Date: 2025-06-18

Audience: [Library Evolution](#)

Target: C++26

Author: [Hana Dusíková](#)

- [Motivation](#)
- [Change](#)
- [Implementation](#)
 - [Reference counting](#)
 - [Exception copying](#)
- [Wording](#)
 - [Feature test macro](#)

Inspecting exception_ptr should be constexpr

Motivation

Since [P3068r6](#) C++ exceptions and most of `exception_ptr` support functions to get `current_exception`, `make_exception_ptr`, and `rethrow_exception` are `constexpr` enabled. But newly added function `exception_ptr_cast` from paper [P2927r3](#) adopted in Sofia is not. I consider this an unfortunate overlook but easily fixable as its implementation is trivial.

Change

This will allow functionality to query if exception is of certain type also during constant evaluation. In terms of wording it's just adding two `constexpr`.

Implementation

I don't have implementation in my library as I learned about it few hours ago, so I have it only here in this paper, but I'm pretty comfortable it will work. There are two possible implementation depending if a platform is having exception storage reference counted. Or is doing copies for each exception rethrow.

Reference counting

```

template <class E> constexpr const E* exception_ptr_cast(const exception_ptr& p) noexcept {
    // Mandates
    static_assert(!std::is_const_v<E>);
    static_assert(!std::is_reference_v<E>);
    static_assert(std::is_object_v<E>);
    static_assert(!std::is_array_v<E>);
    static_assert(!std::is_pointer_v<E>);
    static_assert(!std::is_member_pointer_v<E>);

    try {
        if (p != nullptr) {
            std::rethrow_exception(p);
        }
    } catch (const E & exc) {
        return &exc; // on platforms with ref-counted exceptions
    } catch (...) {
        // fallthrough so we just ignore other exceptions
        // but they are also not copied, just thrown,
        // and still alive in the exception's refcounted storage
    }
    return nullptr;
}

```

Exception copying

In my implementation `exception_ptr` is just a wrapper to `void *` pointer to the exception object. So I can use following implementation:

```

template <class E> constexpr const E* exception_ptr_cast(const exception_ptr& p) noexcept {
    // skipping all the mandates here
    return dynamic_cast<const E *>(p.__ptr); // cute isn't?
}

```

Alternative is to have a builtin to do that information and use metadata known to compiler during constant evaluation in `if constexpr` and do whatever the original paper proposed in runtime. Note constant-time thrown exceptions can't leave constant evaluation according `[expr.const]`.

Wording

In section `[exception.syn]` update definition for `exception_ptr_cast`:

```
constexpr const E* exception_ptr_cast(const exception_ptr& p) noexcept;
```

In section *Exception propagation* [propagation]:

```
constexpr const E* exception_ptr_cast(const exception_ptr& p) noexcept;
```

Mandates: E is a cv-unqualified complete object type. E is not an array type. E is not a pointer or pointer-to-member type. *[Note: When E is a pointer or pointer-to-member type, a handler of type const E& can match without binding to the exception object itself. — end note]*

Returns: A pointer to the exception object referred to by p, if p is not *null* and a handler of type const E& would be a match [except.handle] for that exception object. Otherwise, nullptr.

Feature test macro

17.3.2 Header <version> synopsis [version.syn]

```
#define __cpp_lib_exception_ptr_cast 2026XX202403L // also in <exception>
```